PCIU COMPUTER CLUB

Port City International University · Chattogram, Bangladesh

PCC Website V2.0

Project Handover & Technical Documentation

Document Type	Technical Handover Documentation
Project Name	**PCC Website V2.0**
Prepared For	Authority of the PCIU Computer Club
Date	March 2026

Development Team

Name	Role	Batch
Suprio Das	Team Leader & Backend Developer	CSE 28
Jarin Tasnin	Frontend Developer & UI/UX Designer	CSE 28
Kazi Tajrian Mostafa	UI/UX Designer	CSE __

Prepared by CSE Batch 28 · PCIU Computer Club · Confidential Handover Document

TABLE OF CONTENTS

1. **Executive Summary** . . . 3
2. **Development Team** . . . 3
3. **Technology Stack** . . . 4
   - 3.1 Frontend Technologies . . . 4
   - 3.2 Backend Technologies . . . 4
4. **System Architecture** . . . 5
   - 4.1 High-Level Architecture . . . 5
   - 4.2 Backend MVC Structure . . . 5
5. **Repository Structure** . . . 6
   - 5.1 Frontend Repository . . . 6
   - 5.2 Backend Repository . . . 6
6. **Authentication & Security System** . . . 7
7. **API Documentation** . . . 7
   - 7.1 Resource Groups & Endpoints . . . 8
   - 7.2 HTTP Standards & Response Codes . . . 8
8. **Completed Features & Deliverables** . . . 9
   - 8.1 Backend Deliverables . . . 9
   - 8.2 Frontend Deliverables . . . 9
   - 8.3 Code Quality & DevOps . . . 10
9. **Environment Setup & Local Development** . . . 10
   - 9.1 Prerequisites . . . 10
   - 9.2 Backend Setup . . . 10
   - 9.3 Frontend Setup . . . 11
   - 9.4 Environment Variables Reference . . . 11
10. **Deployment Configuration** . . . 12
11. **Pending Work & Handover Checklist** . . . 12
12. **Important Links & Resources** . . . 13
13. **Guidelines for the Incoming Team** . . . 13

1. Executive Summary

PCC Website V2.0 is the redesigned, re-engineered official website for the PCIU Computer Club (PCC) at Port City International University, Chattogram, Bangladesh. Developed by a dedicated three-member student team from CSE Batch 28, the project replaces the previous club website with a modern, fully-featured, and maintainable full-stack web application.

The system is built on a React + TypeScript frontend and a Node.js + Express.js REST API backend, backed by a MongoDB database. This document is the complete technical and operational handover reference for the incoming developers and club authority.

221	94	TypeScript	Full-Stack MVC
Frontend Git Commits	Backend Git Commits	Primary Language	Architecture

2. Development Team

Suprio Das

Role: Team Leader & Backend Developer

Batch: CSE Batch 28

Key Contributions

- Spearheaded overall project architecture, planning, and team coordination.
- Designed and built the entire backend REST API in Node.js with Express.js.
- Modelled all MongoDB schemas using Mongoose and implemented the full data layer.
- Engineered the JWT + Bcrypt authentication and authorisation system.
- Built and maintained all custom middleware (auth guards, error handlers).
- Managed version control, repository structure, and maintained the Postman API documentation.
- Owned the backend deployment pipeline and environment configuration strategy.

Jarin Tasnin

Role: Frontend Developer & UI/UX Designer

Batch: CSE Batch 28

Key Contributions

- Built all frontend pages and reusable components using React and TypeScript.
- Integrated the frontend with all backend REST API endpoints.
- Implemented client-side routing, state management, and responsive layouts with Tailwind CSS.
- Integrated and customised the shadcn/ui component library for accessible, consistent UI.
- Configured ESLint, Prettier, and Husky to enforce code quality standards across the team.
- Set up and maintained the Vercel deployment pipeline for the frontend.

Kazi Tajrian Mostafa

Role: UI/UX Designer

Batch: CSE Batch 28

Key Contributions

- Defined the overall visual identity, colour palette, typography, and layout language.
- Created wireframes, high-fidelity mockups, and design prototypes for all key pages.
- Ensured a consistent, accessible, and user-friendly design system across the platform.
- Collaborated with the frontend developer to translate designs into functional components.
- Conducted design reviews to maintain visual quality across iterations.

3. Technology Stack

3.1 Frontend Technologies

Category	Technology	Version / Detail	Purpose
Framework	React	v18	Component-based UI library for the single-page application.
Language	TypeScript	v5	Typed superset of JavaScript — 97.5% of the frontend codebase.
Build Tool	Vite	Latest	Lightning-fast dev server with Hot Module Replacement (HMR).
Styling	Tailwind CSS	v3	Utility-first CSS framework for responsive, consistent design.
UI Components	shadcn/ui	Latest	Accessible component library built on Radix

			UI primitives.
Code Formatter	Prettier	Configured	Automated code formatting for consistent style across files.
Linter	ESLint	Configured	Static analysis tool to catch bugs and enforce code standards.
Git Hooks	Husky	Configured	Runs lint and format checks automatically before every commit.
Deployment	Vercel	Production	Zero-config frontend hosting with GitHub CI/CD integration.
Package Manager	npm	Bundled	Dependency management and script runner.

3.2 Backend Technologies

Category	Technology	Version / Detail	Purpose
Runtime	Node.js	v18+	Server-side JavaScript runtime environment.
Framework	Express.js	v4	Lightweight, flexible framework for building REST APIs.
Database	MongoDB	v6	NoSQL document-oriented database for flexible data storage.
ODM	Mongoose	v8	Schema-based ODM for MongoDB with validation and type casting.
Authentication	JSON Web Token	Latest	Stateless, signed token-based authentication.
Security	Bcrypt	Latest	One-way password hashing — no plain-text credentials stored.
Cookies	Cookie-Parser	Latest	Middleware for parsing HTTP cookie headers on the server.
Configuration	Dotenv	Latest	Loads environment variables from .env into process.env.
Language	JavaScript	ES2022	100% JavaScript backend codebase.

4. System Architecture

4.1 High-Level Architecture

PCC Website V2.0 follows a decoupled client-server architecture. The frontend (React SPA) and the backend (Express API) are maintained as separate repositories and communicate exclusively through a RESTful JSON API over HTTPS. This separation makes each layer independently deployable, testable, and maintainable.

Layer	Technology	Responsibility
Client (Browser)	`React + TypeScript`	Renders the UI, handles user interaction, and calls the REST API.
Frontend Hosting	`Vercel CDN`	Serves the compiled React SPA as static files globally.
API Gateway	`Express.js Router`	Receives HTTP requests and dispatches them to the correct controller.
Business Logic	`Controllers`	Validates input, processes data, and formulates JSON responses.
Data Access Layer	`Mongoose Models`	Abstracts MongoDB operations through schema-driven models.
Database	`MongoDB`	Persists all application data as BSON documents in collections.

4.2 Backend MVC Structure

The backend strictly follows the Model-View-Controller (MVC) pattern adapted for an API context — the 'View' is the JSON response returned to the client. Each layer has a single, clear responsibility:

`Controllers/` The brain of the API. Each file handles a specific resource's business logic — receiving the request, executing operations (using Models), and sending the response.

`Models/` Mongoose schema definitions. Each file maps to one MongoDB collection (e.g., User, Event, Notice, Gallery, Member, Executive).

`Routes/` Express Router files mapping HTTP method + URL patterns to their controller functions.

`Middlewares/` Pipeline functions that execute before route handlers — primarily JWT verification, role checks, and request validation.

`Utils/` Shared helper functions used across the codebase — token generators, error formatters, and other reusable utilities.

`index.js` The entry point: initialises the Express app, connects to MongoDB, registers all routes, and starts the HTTP listener.

5. Repository Structure

5.1 Frontend Repository — PCC-V2.0

GitHub URL: https://github.com/Suprio-Das/PCC-V2.0

Language: TypeScript (97.5%) | CSS (2%) | Other (0.5%)

Commits: 221 commits on main branch

`.husky/`	Git hook scripts (pre-commit: runs ESLint + Prettier automatically)
`public/`	Static assets served directly — favicon, OG images, robots.txt
`src/`	All application source code
`├─ components/`	Reusable React UI components (buttons, cards, navbars, modals, etc.)
`├─ pages/`	Page-level route components — one per application route
`├─ hooks/`	Custom React hooks for shared stateful logic
`├─ lib/`	API client setup, utility functions, and helper modules
`├─ types/`	Global TypeScript type definitions and interface declarations
`└─ assets/`	Images, icons, and other static resources bundled by Vite
`.eslintrc.cjs`	ESLint configuration — rules and parser settings
`.prettierrc`	Prettier formatting rules
`components.json`	shadcn/ui component registry and theming configuration
`index.html`	Root HTML template and Vite application entry point
`package.json`	NPM dependency manifest and project scripts (dev, build, lint)
`tailwind.config.ts`	Tailwind CSS theme extensions and plugin configuration
`tsconfig.json`	TypeScript compiler configuration
`vite.config.ts`	Vite bundler configuration with path aliases
`vercel.json`	Vercel deployment config — rewrites all routes to index.html for SPA

5.2 Backend Repository — PCC-V2.0-Backend

GitHub URL: https://github.com/Suprio-Das/PCC-V2.0-Backend

Language: JavaScript (100%)

Commits: 94 commits on main branch

`Controllers/`	Business logic handler functions for each API resource

`Middlewares/`	JWT auth verification middleware and other request pipeline functions
`Models/`	Mongoose data schemas defining the structure of each MongoDB collection
`Routes/`	Express Router files — HTTP method + path → controller function mappings
`Utils/`	Shared utility/helper functions used across controllers and middleware
`index.js`	App entry point — Express init, MongoDB connection, route registration, server start
`package.json`	NPM dependencies and scripts
`.gitignore`	Git exclusions — includes .env, node_modules, and sensitive files

6. Authentication & Security System

PCC V2.0 implements a production-grade, stateless authentication system. It combines JSON Web Tokens (JWT) for session management with Bcrypt for password security, following industry-standard security practices throughout.

Authentication Flow

Registration: User submits credentials → password is bcrypt-hashed (salted, never stored plain) → user document saved to MongoDB → signed JWT returned.

Login: User submits email + password → backend queries the user document → bcrypt.compare() verifies the password hash → a new signed JWT is issued.

Token Delivery: The JWT is delivered via Cookie-Parser as an HTTP cookie (configurable) or in the JSON response body — consumed by the frontend.

Authenticated Request: Frontend attaches the JWT in every protected request header as Authorization: Bearer <token>.

Middleware Verification: The auth middleware intercepts protected routes → verifies the JWT signature and expiry using the JWT_SECRET → decodes user identity → attaches it to req.user → passes to the controller.

Rejection: Any request with a missing, malformed, or expired token receives a 401 Unauthorized response — the route handler is never reached.

Security Practices Implemented

- Passwords are hashed using bcrypt with salt rounds — reversing the hash is computationally infeasible.
- The JWT signing secret (JWT_SECRET) is stored exclusively as an environment variable — never in source code.
- JWTs include an expiry claim (exp) to limit the exposure window of any compromised token.
- All sensitive API routes are gated behind the auth middleware — unauthenticated access is impossible at the route level.

- The .env file is explicitly excluded from Git via .gitignore — no secrets have ever been committed to the repository.
- Cookie-Parser is used to enable HTTP-only cookie delivery, preventing JavaScript-based token theft.

7. API Documentation

> *The complete, interactive API documentation is available on Postman at: https://documenter.getpostman.com/view/40171504/2sBXigMtnb*

The entire backend REST API is documented, organised, and testable through a Postman collection. All endpoints follow RESTful conventions — standard HTTP methods, predictable URL patterns, JSON request/response bodies, and appropriate HTTP status codes.

Base URL (Development): http://localhost:[PORT]

Base URL (Production): https://[your-backend-domain]/

Content-Type Header: application/json

Auth Header (Protected): Authorization: Bearer <JWT_TOKEN>

7.1 Resource Groups & Endpoints

Resource Group	Base Route	Key Operations
Auth / Users	/api/auth	Register, Login, Logout, Get Profile, Update Profile
Events	/api/events	Create Event, List All, View Single, Update, Delete
Notices	/api/notices	Post Notice, List All, View Single, Update, Delete
Gallery	/api/gallery	Upload Media Entry, List Gallery, View Item, Delete
Members	/api/members	Add Member, List Members, View Profile, Update, Delete
Executives	/api/executives	Add Executive, List Committee, Update Role, Delete

7.2 HTTP Standards & Response Codes

Status Code	Meaning	When Returned

200 OK	Success	Successful GET, PUT, or PATCH request.
201 Created	Resource Created	Successful POST — a new document was inserted.
400 Bad Request	Client Error	Malformed request body or missing required fields.
401 Unauthorized	Auth Required	No JWT token, or the token is invalid/expired.
403 Forbidden	Access Denied	Valid token but insufficient role/permissions.
404 Not Found	Missing Resource	The requested document does not exist.
500 Internal Server Error	Server Error	Unexpected server-side failure.

8. Completed Features & Deliverables

The following section is the definitive record of everything built, tested, and delivered by the development team. All items listed here are complete and ready for the incoming authority to review, deploy, and extend.

8.1 Backend Deliverables

✦ **Express.js Server & Application Bootstrap**

A fully configured Express.js application in index.js. It initialises the server, connects to MongoDB via Mongoose, registers all route groups, attaches global middleware (JSON parsing, CORS, Cookie-Parser), and starts the HTTP listener on the configured port.

✦ **MongoDB Database Layer with Mongoose Schemas**

Complete Mongoose schema definitions for all major data entities: Users, Events, Notices, Gallery items, Members, and Executive Body members. Each schema includes field-level validation, type definitions, default values, and timestamps — ensuring data integrity at the application layer.

✦ **Full CRUD REST API for All Resources**

RESTful endpoints implementing Create, Read, Update, and Delete operations for every resource group (Auth/Users, Events, Notices, Gallery, Members, Executives). All routes are mapped in the Routes/ folder and handled by dedicated controller functions.

✦ **JWT + Bcrypt Authentication System**

A complete, stateless authentication pipeline: user registration with bcrypt-hashed passwords, JWT-based login, secure token issuance, and protected route enforcement through the auth middleware — all working end-to-end.

✦ Middleware Pipeline

Custom middleware stack including: JWT verification middleware (guards all protected routes), global error handling middleware (catches unhandled errors and returns clean JSON responses), and Cookie-Parser integration for cookie-based token delivery.

✦ Utility Layer

A collection of reusable helper utilities in the Utils/ folder — including JWT generation functions, standardised error response formatters, and other shared logic extracted from controllers for clean separation of concerns.

✦ Environment-Based Configuration

All sensitive configuration (database URI, JWT secret, port) is managed through environment variables using dotenv. The .gitignore is configured to exclude .env from version control — no secrets exist in the repository history.

✦ Postman API Documentation

The entire API surface is documented in a Postman collection covering all endpoints, request formats, required headers, authentication requirements, and example responses. Available publicly at the Postman documentation URL.

8.2 Frontend Deliverables

✦ React + TypeScript Project Foundation

A fully bootstrapped, production-ready Vite + React + TypeScript project. TypeScript is configured with strict type-checking across the entire codebase (tsconfig.json), ensuring type safety at compile time. Vite provides a sub-second development server with HMR and optimised production builds.

✦ Tailwind CSS Design System

Tailwind CSS is fully integrated and configured with a custom theme in tailwind.config.ts, matching the visual identity designed by the UI/UX team. The design system is consistent, responsive by default, and easily extensible by future developers.

✦ shadcn/ui Component Library Integration

The shadcn/ui component library is integrated and configured via components.json. This provides the application with a set of accessible, composable, and customisable UI primitives (dialogs, buttons, inputs, dropdowns, etc.) built on Radix UI — ensuring WCAG accessibility standards are met out of the box.

✦ Page Architecture & Client-Side Routing

All application pages are structured in src/pages/ with React Router managing client-side navigation. The vercel.json deployment configuration ensures all URL paths are correctly redirected to index.html for SPA routing to function properly in production.

✦ Reusable Component Library

> A comprehensive library of reusable React components in src/components/ — covering layout elements (Navbar, Footer, Sidebar), content display (EventCard, MemberCard, NoticeCard, GalleryItem), form elements, and other shared UI building blocks.

✦ **Backend API Integration Layer**

> The frontend is fully connected to all backend API endpoints. All data flows — fetching events, notices, gallery items, member data, and executive listings — are wired through a structured API client in src/lib/. Authentication headers and token management are handled transparently for all authenticated requests.

✦ **Vercel Production Deployment**

> The frontend is configured for and deployable on Vercel with zero additional configuration. The vercel.json file handles SPA routing, and the project builds cleanly with npm run build, outputting an optimised static bundle to the dist/ directory.

8.3 Code Quality & DevOps

Practice / Tool	What It Does & Its Value
ESLint	Configured with .eslintrc.cjs for static code analysis — enforces consistent coding patterns and catches common bugs before runtime.
Prettier	Configured with .prettierrc for automatic, opinionated code formatting — eliminates all style debates and ensures a uniform codebase.
Husky Git Hooks	Pre-commit hooks run ESLint and Prettier automatically before any commit is accepted — bad code cannot enter the repository.
.prettierignore	Specific files and directories are excluded from formatting where appropriate (e.g., generated files, build output).
TypeScript Strict Mode	The frontend compiler is configured for strict type-checking — catching type mismatches, null reference errors, and implicit any types at build time, not runtime.
Git Version Control	Both repositories maintain clean, linear Git history across 315 total commits (221 frontend + 94 backend), providing full traceability of every change made during development.

9. Environment Setup & Local Development

9.1 Prerequisites

Prerequisite	Version	Purpose
Node.js	v18 or later	Runtime for both running the backend and building the frontend.
npm	Bundled with Node.js	Package manager — install dependencies with npm install.
MongoDB	Local or Atlas	Database — local Community Edition or a free MongoDB Atlas cluster.
Git	Latest	Clone both repositories and manage version control.
VS Code	Recommended	Code editor — install the ESLint and Prettier extensions.

9.2 Backend Setup

Step 1 — Clone the repository

```
git clone https://github.com/Suprio-Das/PCC-V2.0-Backend.git
cd PCC-V2.0-Backend
```

Step 2 — Install all dependencies

```
npm install
```

Step 3 — Create the environment file

```
# Create a file named .env in the project root (see Section 9.4)
```

Step 4 — Start the development server

```
npm start
# Server will listen on http://localhost:[PORT]
```

9.3 Frontend Setup

Step 1 — Clone the repository

```
git clone https://github.com/Suprio-Das/PCC-V2.0.git
cd PCC-V2.0
```

Step 2 — Install all dependencies

```
npm install
```

Step 3 — Create the environment file

```
# Create a .env file in the project root (see Section 9.4)
```

Step 4 — Start the development server

```
npm run dev
# App will run at http://localhost:5173 (Vite default)
```

Step 5 — Build for production

```
npm run build
# Outputs an optimised static bundle to dist/
```

9.4 Environment Variables Reference

Variable	Required In	Description & Example Value
PORT	Backend	Port for the Express server. Example: 5000
MONGO_URI	Backend	MongoDB connection string. Example: mongodb://localhost:27017/pcc or a MongoDB Atlas URI.
JWT_SECRET	Backend	Long random string for signing JWT tokens. Must be secret. Example: my_super_secret_key_xyz
NODE_ENV	Backend	Runtime environment. Values: development or production
VITE_API_BASE_URL	Frontend	Full URL of the backend API. Example: http://localhost:5000 (dev) or https://api.pcc.example.com (prod)

> *Important: Never commit your .env files to Git. For production, set these variables through your hosting platform's environment settings dashboard.*

10. Deployment Configuration

Frontend — Vercel

The frontend is configured for Vercel deployment. The vercel.json file in the root of PCC-V2.0 handles SPA routing by redirecting all URL paths to index.html, which is required for React Router to manage client-side navigation in production.

Setting	Value
Build Command	npm run build
Output Directory	dist

Framework Preset	Vite (auto-detected)
Env Variable	VITE_API_BASE_URL = [live backend URL]
Auto-Deploy	Enabled on every push to main branch

Backend — Recommended Hosting Platforms

Platform	URL	Notes
Railway	`railway.app`	Recommended. GitHub integration, auto-deploy, supports Node.js + MongoDB add-on.
Render	`render.com`	Free tier available. Simple Git-based deploy with env variable management.
AWS / GCP	`Cloud provider`	For future production-grade, highly scalable deployment.

Database — MongoDB Atlas

For production, use MongoDB Atlas (mongodb.com/cloud/atlas) — a fully managed cloud database with a generous free tier. Create a cluster, set up a database user, whitelist the backend server's IP, and copy the connection string into the MONGO_URI environment variable.

11. Pending Work & Handover Checklist

The following items are outstanding tasks recommended for the incoming team to complete before or shortly after the public launch. They are listed by priority.

Pre-Launch (Critical)

- Set up the production MongoDB Atlas cluster and configure MONGO_URI.
- Deploy the backend to a hosting platform (Railway / Render) with all environment variables set.
- Update VITE_API_BASE_URL on Vercel to the live backend URL and trigger a re-deploy.
- Configure CORS on the backend to allow only the production frontend origin.
- End-to-end test all API endpoints against the live frontend before going public.
- Rotate JWT_SECRET immediately after receiving repository access.
- Implement file/image upload support (Cloudinary / AWS S3) for the Gallery module.
- Add server-side input validation on all POST/PUT routes (express-validator or Joi).

Post-Launch Enhancements

- Add rate limiting (express-rate-limit) to protect public API endpoints.
- Implement password reset via email (Nodemailer + SendGrid).
- Add pagination to all list endpoints for scalability.
- Set up automated MongoDB Atlas database backups.
- Write integration tests for critical backend routes (Jest + Supertest).

12. Important Links & Resources

Resource	URL
Frontend Repository	https://github.com/Suprio-Das/PCC-V2.0
Backend Repository	https://github.com/Suprio-Das/PCC-V2.0-Backend
API Documentation (Postman)	https://documenter.getpostman.com/view/40171504/2sBXigMtnb
Vercel — Frontend Hosting	https://vercel.com
MongoDB Atlas	https://www.mongodb.com/cloud/atlas
Railway — Backend Hosting	https://railway.app
shadcn/ui Documentation	https://ui.shadcn.com
Tailwind CSS Docs	https://tailwindcss.com/docs
Mongoose Documentation	https://mongoosejs.com/docs
Express.js Documentation	https://expressjs.com
JWT Introduction	https://jwt.io/introduction

13. Guidelines for the Incoming Team

Getting Started

- Read this entire document before opening any code — it will save significant time.
- Clone both repositories and complete the local setup in Section 9 before anything else.

- Import the Postman collection (Section 12) and run every endpoint to understand the API surface.
- Trace one full data flow: from a frontend page → API call → Express route → Controller → Mongoose Model → MongoDB → response.
- Review the Git commit history on both repositories — it documents the entire development journey.

Development Best Practices

- Always branch from main — use descriptive names like feature/admin-dashboard or fix/event-404-bug. Never commit directly to main.
- Write clear, meaningful commit messages — e.g., 'Add JWT expiry to auth middleware' not 'fix'.
- Husky will reject commits that fail ESLint or Prettier — do not bypass these hooks.
- Test all backend changes in Postman before connecting to the frontend.
- Update the Postman documentation for any new or modified endpoints.
- Never commit .env files — set environment variables through the hosting platform's dashboard.

Ownership Transfer Checklist

- Transfer both GitHub repositories to the official PCC organisation account.
- Rotate the JWT_SECRET in the production environment immediately after handover.
- Update MongoDB Atlas credentials and regenerate the MONGO_URI connection string.
- Update the VITE_API_BASE_URL on Vercel to reflect the new production backend URL.
- Add new team members as collaborators to both GitHub repositories.
- Schedule regular MongoDB Atlas database backups.

A Note from the Development Team

We — Suprio Das, Jarin Tasnin, and Kazi Tajrian Mostafa — built PCC Website V2.0 with genuine care for the PCIU Computer Club and its members. The foundation is solid: a modern stack, clean architecture, and a fully functional API. We hope this platform serves the club well for years to come. Build something great.

— Team PCC V2.0 · CSE Batch 28 · March 2026